

Exercise 8: The Debugging Loop and Checkpoints

DEV CYCLE	Fix (debugging)
CC FEATURES	The debugging loop (full error + context), checkpoints and rewind (Esc Esc, /rewind), targeted fixes
SPEC TRACES	FR-20 (review queue)
TIME	25 min
YOU PRODUCE	A deliberately broken review-queue endpoint, diagnosed and fixed through the loop

Objective

AI-generated code breaks in predictable ways, usually at the edges: the empty list, the first item, the limit. You will introduce a realistic bug in the reviewer's work queue, then run the debugging loop properly: full error plus context, not just the last line. You will also learn checkpoints and rewind, which let you undo an AI change that made things worse.

Walk-through

1 Plant a realistic bug

Paste this route into `app/routes.py` yourself, as-is. It mirrors what under-specified AI code looks like: it works on the happy path you tried once, and fails everywhere else. Make sure `datetime`, `timezone`, `SummaryDraft` and `DraftOut` are imported.

CREATE / PASTE → APP/ROUTES.PY (ADD THIS BROKEN ROUTE)

```
@router.get("/review-queue")
def review_queue(limit: int = 10, db: Session = Depends(get_db)) -> dict:
    """FR-20: oldest drafts awaiting review. (Contains bugs on purpose.)"""
    rows = (db.query(SummaryDraft)
            .filter(SummaryDraft.status == "pending_review")
            .order_by(SummaryDraft.created_at.asc())
            .all())
    # BUG 1: assumes rows[0] exists. BUG 2: off-by-one slice.
    oldest = rows[0].created_at
    wait_h = (datetime.now(timezone.utc).replace(tzinfo=None) - oldest).total_seconds() /
3600
    return {"oldest_wait_hours": round(wait_h, 1),
            "items": [DraftOut.model_validate(r) for r in rows[: limit - 1]]}
```

VALIDATE The server reloads. With two or more pending drafts, `GET /review-queue?limit=2` returns only one item (off-by-one). With no pending drafts it returns 500 and an `IndexError` in the uvicorn console.

2 Reproduce and capture the full error

Force the empty case: approve or reject every pending draft in Swagger (or stop the server, delete `dischargeflow.db`, and restart). Then call the endpoint and copy the entire traceback from the uvicorn terminal. The discipline: never paste just the last line.

WINDOWS · POWERSHELL (SECOND TERMINAL)

```
PS> Invoke-RestMethod "http://127.0.0.1:8000/review-queue?limit=2"
```

MACOS / LINUX · BASH OR ZSH

```
$ curl -s "http://127.0.0.1:8000/review-queue?limit=2"
```

VALIDATE You get a 500 (Internal Server Error) and a Python traceback ending in `IndexError: list index out of range` in the uvicorn terminal.

3 Run the debugging loop

Paste the full traceback into Claude with context about what you were doing and what the spec says should happen. Context is what turns a guess into a diagnosis.

TYPE THIS PROMPT INTO CLAUDE CODE

GET /review-queue (FR-20 in @docs/SPEC.md) returns 500 when no drafts are pending, and one fewer item than the limit when there are. Here is the full traceback and the route. What are the bugs, and what is the minimal fix that satisfies FR-20 exactly? Do not touch any other route. [paste the whole traceback and the route here]

VALIDATE Claude identifies both bugs: the unguarded `rows[0]` and the `limit - 1` slice. It proposes returning `oldest_wait_hours: null` with an empty list, and slicing `[ :limit]`, ideally in the query with `.limit(limit)`. It may also notice FR-20 requires $1 \leq \text{limit} \leq 50$. It should NOT propose rewriting unrelated code.

4 Apply, then deliberately rewind

Let Claude apply the fix. Then practise the safety net: press Esc twice (or run `/rewind`) to open the checkpoint menu. Claude Code saves a checkpoint before each change, and you can restore the code, the conversation, or both.

INSIDE THE CLAUDE CODE SESSION

```
# Press Esc twice, or:  
> /rewind
```

VALIDATE The menu lists checkpoints from this session, including the point just before the fix. Close the menu (or restore forward) so the fix stays applied. You now know how to undo a bad AI edit in seconds.

SCREENSHOT SLOT *The rewind menu listing checkpoints before and after the fix*

5 Confirm the fix against the spec

Re-run every case that was broken, plus the spec boundaries.

SECOND TERMINAL

```
PS> Invoke-RestMethod "http://127.0.0.1:8000/review-queue?limit=2"  
# create 3 pending drafts via Swagger (3 records), then:  
PS> Invoke-RestMethod "http://127.0.0.1:8000/review-queue?limit=2"  
PS> Invoke-RestMethod "http://127.0.0.1:8000/review-queue?limit=0"
```

VALIDATE Empty queue: 200 with `items: []` and `oldest_wait_hours: null`, no 500. Three pending drafts with `limit=2`: exactly two items, oldest first. `limit=0` (and `limit=51`): 422, per FR-20. All three original failures are gone.

What success looks like

- You reproduced a bug, captured the full traceback, and fixed it via the loop rather than by trial and error.
- The fix was minimal, targeted, and measured against the spec, not against "it works now".
- You can open the rewind menu and explain what checkpoints do and do not protect you from.

Common pitfalls (Windows and macOS)

- Pasting only the last line of a traceback is the number-one debugging mistake. The frames above it are where the answer lives.
- Esc Esc is the rewind shortcut; a single Esc just interrupts the current turn. They are different.
- Checkpoints cover edits Claude made in this session. They do not undo commands like deleting the database, and they are not a substitute for git (Exercise 11).
- Naive vs timezone-aware datetimes: SQLite returns naive values. If you see `can't subtract offset-naive and offset-aware datetimes`, that is a third bug worth a sentence in your ADR.

Go deeper • AI developer track

Optional, for faster finishers. Each challenge extends the lab with a production-grade technique.

- Add request-ID middleware and structured JSON logs (no PHI), then re-run the debugging loop with the request ID in your prompt. Compare how quickly Claude localises the fault.
- Seed a regression across three commits and use `git bisect run python -m pytest -q -x` with Claude driving. Explain what makes a test suitable as a bisect oracle.

Stretch goal

Ask Claude: "what other inputs could break this endpoint?" Then turn each answer into a test: empty queue, limit at 1 and 50, limit 0 and 51, drafts created in the same second. You are turning a bug into a permanent guard, which is the bridge to the next exercise.